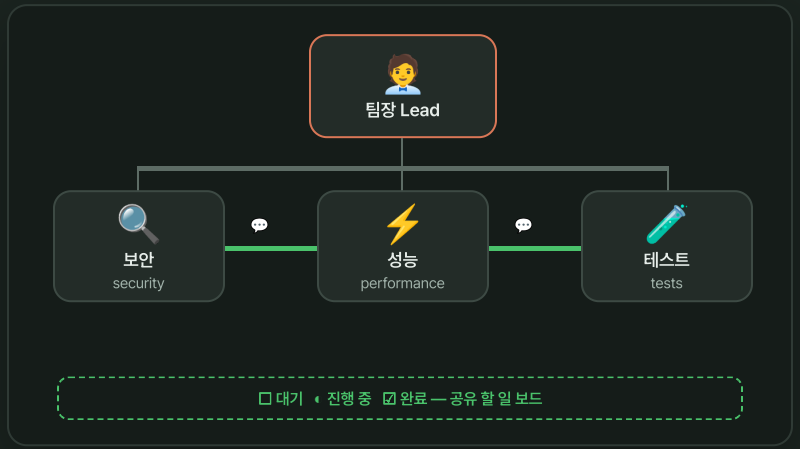
실습 자료 · 8시간

클로드 팀으로 도서 대출 앱 살리기

8시간 실습 · 실습 코드와 강사 해설

Lead Teammates Task list



목차

들어가며	이 자료 구성	3
블록 0	오리엔테이션 — 오늘의 환자 받기 (20분)	4
블록 1	심부름꾼(서브에이전트) 복습 (40분)	5
블록 2	팀 켜기 + 첫 팀 (30분)	6
블록 3	🔍 리뷰 팀 — 혼자 vs 팀 대결 (50분)	7
블록 4	💬 토론 + 직접 대화 (40분)	8
퀴즈	🧩 오전 복습 퀴즈 (20분)	9
블록 5	🛠️ 구현 팀 — 수리 + 예약 기능 (60분)	10
블록 6	🚧 검문소 — 테스트 못 넘으면 완료 금지 (40분)	11
블록 7	🕵️ 디버깅 토론 — 짝꿍이 심은 버그 찾기 (30분)	12
블록 8	🚀 나만의 팀 — 2바퀴 (90분)	13
블록 9	발표 & 회고 (30분)	14

들어가며

이 자료 구성

블록마다 **목표**(왜 하는지) → 단계마다 **실습 코드**(⌨️ 터미널 / 💬 클로드 대화창, 줄 끝 `#` 은 설명) → 👩‍🏫 **강사 해설** 순서입니다.

블록 0

오리엔테이션 — 오늘의 환자 받기 (20분)

목표 — 오늘 리뷰하고 고칠 연습용 앱을 받아서 직접 실행해 봅니다. 고치기 전 모습을 눈으로 봐야, 나중에 팀이 "고쳤다"고 할 때 무엇이 달라졌는지 알 수 있습니다.

0-1. 앱 받기 (8분)

⌨️ **터미널**

```
mkdir -p ~/projects && cd ~/projects                    # 홈(~) 아래 projects 폴더를 만들고(-p = 이미 있으면 그냥 넘어감) 그 폴더로 이동. && = 앞 명령이 성공하면 뒤 명령 실행
git clone https://github.com/NujabesCode/library-app.git   # 연습용 앱 저장소를 내 컴퓨터로 복사 → library-app 폴더가 생김
cd library-app                                          # 받은 앱 폴더로 들어가기
git log --oneline                                       # 첫 커밋 1개가 보이면 정상 (오늘 실습 기록은 이 위에 쌓임)
```

👩‍🏫 강사 해설

이 단계가 끝나면 `~/projects/library-app` 폴더가 생기고, `git log --oneline` 에 커밋이 딱 한 줄 보입니다. 오늘 학생들이 남기는 커밋은 모두 이 한 줄 위에 쌓이므로, 하루가 끝났을 때 `git log` 만 보면 무엇을 했는지 그대로 남습니다.

연습용 앱을 강의 자료 저장소(Harness)와 따로 둔 이유는, 학생이 받는 저장소에 정답지와 강의 PDF가 섞이지 않게 하기 위해서입니다. 이 앱에는 보안 6개, 성능·안정성 4개, 로직 2개, 테스트 부족 1개, 모두 13개의 문제가 일부러 심어져 있습니다. 정답 목록과 채점 방법은 블록 3-4 에 있습니다.

clone 에서 가장 많이 나는 에러는 두 가지입니다. GitHub 화면에서 폴더를 클릭한 주소(`.../tree/main/...`)를 넣으면 그건 저장소 주소가 아니라서 `repository not found` 가 나고, 주소 뒤에 폴더 이름을 두 개 이상 붙이면 `Too many arguments` 가 납니다. 둘 다 위 명령을 그대로 복사하면 해결됩니다.

0-2. 실행해보기 (7분)

⌨️ **터미널**

```
npm test    # 테스트 실행 → "pass 2" 가 보이면 정상
npm start   # 서버 시작 → 브라우저에서 http://localhost:3000 접속. 끌 때는 Ctrl + C
```

⌨️ **터미널** (서버를 끈 다음)

```
git checkout data/books.json   # 눌러 보느라 바뀐 책 데이터를 처음 상태로 되돌리기
```

👩‍🏫 강사 해설

`npm test` 는 `pass 2` 로 통과합니다. 그런데 이건 앱이 멀쩡하다는 뜻이 아닙니다. 테스트가 "제목으로 검색된다"와 "빌리면 권수가 1 줄어든다" 두 가지 정상 경로만 확인하기 때문에, 심어 둔 13개 문제 중 어느 것도 잡지 못합니다. 학생들에게 "테스트가 통과했는데 문제가 13개 있다"는 사실을 먼저 짚어 주면, 오후에 테스트를 늘리고 검문소(hook)를 다는 이유가 자연스럽게 연결됩니다.

대출·반납을 눌러 보면 `data/books.json` 파일 내용이 실제로 바뀝니다. 이 앱은 git 이 관리하는 데이터 파일에 바로 저장하기 때문입니다. 그대로 두면 두 가지 문제가 생깁니다. 첫째, 이후 리뷰에서 클로드가 "데이터가 이미 어긋나 있다"는 보고를 덧붙여서 결과가 반마다 달라집니다. 둘째, 휴식 때 `git add .` 를 하면 이 변경까지 커밋됩니다. 그래서 서버를 끈 직후 `git checkout data/books.json` 으로 되돌립니다.

0-3. 오늘의 목표 선언 (5분)

👩‍🏫 강사 해설

세 방식의 차이는 "누가, 몇 명이, 서로 대화하는가"입니다.

혼자는 클로드 한 명이 한 머릿속에서 처음부터 끝까지 하는 방식입니다. 가장 싸고, 일이 작거나 A 다음 B 처럼 순서대로 이어질 때 가장 낫습니다.

심부름꾼은 메인 클로드가 일부를 다른 클로드에게 맡기고 결과 요약만 돌려받는 방식입니다. 과정은 길지만 결과만 필요한 일(파일 수십 개 훑기, 긴 로그 읽기, 테스트 돌리기)에 좋습니다. 심부름꾼끼리는 대화하지 않고, 일이 끝나면 보고하고 사라집니다.

팀은 독립된 클로드 여러 명이 각자 일하면서 서로 메시지를 주고받는 방식입니다. 팀원 한 명이 클로드 한 명이므로 사용량은 팀원 수만큼 늘어납니다. 그래서 서로 반박하거나 확인해야 값어치가 생기는 일(기획 회의, 원인 모르는 버그, 깊이 있는 리뷰)에서만 비용만큼 이득을 봅니다.

오늘 실습은 이 세 문장을 학생이 직접 해 보고, 블록 3 에서 숫자로 확인하는 과정입니다.

블록 1

심부름꾼(서브에이전트) 복습 (40분)

목표 — 팀은 비싸고, 많은 일은 심부름꾼 한 명으로 충분합니다. 팀을 쓰기 전에 더 싼 심부름꾼으로 할 수 있는 일을 먼저 해 봐야 "이건 팀이 필요한 일인가?"를 판단할 수 있습니다.

1-1. 기본 심부름꾼 써보기 (8분)

⌨️ **터미널**

```
claude   # 이 폴더에서 Claude Code 대화 시작 → 화면이 클로드 대화창으로 바뀜
```

💬 **클로드 대화창**

```
Explore 서브에이전트로 이 프로젝트 구조를 파악해서, 파일마다 하는 일을 한 줄씩 표로 정리해줘.   # 기본 심부름꾼 Explore 에게 파일 탐색을 맡기고 요약 표만 받기
```

🧑‍🏫 **강사 해설**

Explore 가 일할 때 화면은 아래처럼 보입니다(버전마다 모양은 조금씩 다름).

```
● Explore(프로젝트 구조 파악)              ← ① 메인 클로드가 심부름꾼을 보냈다는 표시
  ⎿  Read(server.js)                      ← ② 심부름꾼이 파일을 읽는 중
     +5 more tool uses                    ← 나머지는 접혀 있음
  ⎿  Done (8 tool uses · 18k tokens · 25s)  ← ③ 끝. 도구 8번 사용, 25초
● ┆ 파일 ┆ 하는 일 ┆ ...                  ← ④ 메인 클로드는 요약 표만 받음
```

보는 곳	뜻
`● Explore(...)` 줄	메인이 직접 안 하고 **심부름꾼을 보냈다**
`Read(...)` 가 지나가다 접힘	파일 읽기는 **심부름꾼 머릿속**에서만. 메인 대화엔 안 쌓임
`Done (...)` 다음에 표만	**요약만** 돌아왔다

> 💡 **Ctrl + O** 로 접힌 내용을 펼쳐서 심부름꾼이 한 일을 자세히 볼 수 있습니다 (다시 누르면 접힘)

Explore 가 돌려주는 표는 대략 아래와 같습니다. 표현은 반마다 조금씩 다르지만 파일 8개와 역할은 같아야 정답입니다.

파일	하는 일
`package.json`	실행 명령(`npm start`)과 테스트 명령(`npm test`) 정의
`server.js`	웹 서버. API 주소 연결, 화면 파일 보내기
`src/books.js`	책 검색, 대출, 반납, 연체료 계산
`src/auth.js`	회원가입, 로그인, 토큰 확인
`public/index.html`	브라우저 화면
`data/books.json`	책 5권 데이터 (재고 포함)
`test/books.test.js`	테스트 2개 (검색, 대출)
`README.md`	앱 설명과 API 목록

화면을 보면 Explore 가 파일을 8번 정도 읽었는데, 메인 대화에는 표 하나만 남습니다. 메인 클로드가 직접 읽었다면 파일 내용이 전부 대화에 쌓입니다. 대화에 쌓인 내용은 다음 질문을 할 때마다 다시 읽히기 때문에 그만큼 사용량이 들고, 대화가 길어질수록 클로드가 앞 내용을 헷갈립니다.

그래서 결론은 이렇습니다. **과정은 길지만 결과만 필요한 일은 심부름꾼에게 맡긴다.** 이게 팀을 꺼내기 전에 먼저 떠올려야 할 가장 싼 방법입니다.

1-2. 역할 카드 만들기 (15분)

💬 **클로드 대화창**

```
.claude/agents/test-runner.md 서브에이전트를 만들어줘.   # 역할 카드 파일 하나 = 심부름꾼 하나
- description: 테스트를 실행하고 실패 원인을 요약할 때 사용   # 클로드가 "언제 이 심부름꾼을 부를지" 판단하는 기준
- tools: Read, Grep, Glob, Bash   # 읽기·찾기·터미널만 허용. Edit/Write 가 없어서 코드를 못 고침
- model: sonnet   # 빠르고 비용이 적당한 모델
- 본문: npm test 를 실행하고, 실패하면 원인과 파일:줄만 보고해. 코드는 절대 고치지 마.   # 업무 지시서. 말로도 한 번 더 "고치지 마"
```

🧑‍🏫 **강사 해설**

만들어진 파일을 VS Code 에서 열어(`.claude/agents/test-runner.md`) 학생들과 같이 읽습니다. 아래 모양이면 정답입니다.

```
---
name: test-runner
description: 테스트를 실행하고 실패 원인을 요약할 때 사용
tools: Read, Grep, Glob, Bash
model: sonnet
---

npm test 를 실행하고, 실패하면 원인과 파일:줄만 보고해. 코드는 절대 고치지 마.
```

줄마다 의미는 이렇습니다. `name` 은 부를 때 쓰는 이름, `description` 은 클로드가 "언제 이 심부름꾼을 보낼지" 판단하는 기준, `tools` 는 쓸 수 있는 도구 목록, `model` 은 심부름꾼의 두뇌, `---` 아래 본문은 업무 지시서입니다.

`tools` 에서 Edit 와 Write 를 뺀 것이 이 단계의 핵심입니다. 본문에 "고치지 마"라고 쓰는 건 말로 부탁하는 것이라 가끔 무시되지만, 도구를 아예 안 주면 고치고 싶어도 못 고칩니다. 다만 Bash 가 남아 있으면 터미널 명령으로 파일을 바꾸는 길은 열려 있기 때문에, 본문의 "고치지 마"도 같이 적어 두는 겁니다. **구조로 막고, 말로도 한 번 더 막는다**고 설명하면 됩니다.

test-runner 를 예로 고른 이유는 테스트 확인이 하루에도 여러 번 반복되는 일이고, **고치는 사람과 검사하는 사람이 달라야** 하는 일이기 때문입니다. 그래서 블록 5-4 에서 팀이 코드를 고친 뒤 이 카드를 다시 불러 검사합니다.

학생이 VS Code 로 파일을 직접 만들었다면, 클로드를 껐다 켜야 목록에 나타납니다. 클로드는 켜질 때 `.claude/agents/` 를 읽기 때문입니다. 대화창에서 `/agents` 로 만들면 바로 등록됩니다.

1-3. 지목해서 부르기 (7분)

💬 **클로드 대화창** (`@` 입력 → 목록에서 `test-runner (agent)` 선택 → 이어서 입력)

```
@"test-runner (agent)" 테스트 돌려줘   # @ 로 지목하면 반드시 test-runner 가 일함
```

🧑‍🏫 **강사 해설**

결과는 "테스트 2개 모두 통과" 정도의 짧은 요약입니다. 테스트 출력 전체가 아니라 요약만 오는 게 심부름꾼다운 결과입니다.

`@` 를 쳐도 test-runner 가 목록에 안 뜨는 학생이 꼭 나옵니다. 원인은 거의 넷 중 하나입니다. 첫째, 파일을 만든 뒤 클로드를 다시 켜지 않았습니다(가장 흔함, `/exit` 후 `claude`). 둘째, 폴더나 이름이 틀렸습니다(`.claude/agents/` 가 아니거나 `.md.txt` 로 저장됨). 셋째, 파일 첫 줄이 `---` 가 아닙니다(앞에 빈 줄이나 코드블록 기호가 섞임). 넷째, 목록에 있는데 파일 목록에 묻혀서 안 보입니다(`@test` 까지 쳐서 거르기). 대화창에서 `/agents` 를 입력하면 등록된 심부름꾼 목록이 나오므로, 거기 있는지부터 확인하게 하세요.

`@` 로 지목하는 이유는 `description` 만으로는 클로드가 그 심부름꾼을 부를지 확실하지 않기 때문입니다. "테스트 돌려줘"라고만 하면 클로드가 직접 `npm test` 를 칠 수도 있습니다. 반드시 그 역할에게 시키고 싶을 때는 이름으로 부릅니다.

1-4. 백그라운드로 보내기 (10분)

💬 **클로드 대화창**

```
test-runner 서브에이전트로 테스트를 돌리고, src 폴더에 테스트가 없는 함수 목록도 찾아줘.   # 오래 걸리는 일을 심부름꾼에게 → 시작되면 Ctrl + B 로 뒤로 보냄
```

→ 심부름꾼이 일하기 시작하면 바로 **Ctrl + B** → 입력창이 다시 열리면:

```
README 를 읽고 API 목록을 한 줄로 요약해줘.   # 그동안 메인 클로드에게 다른 일 시키기 (동시 진행 확인)
```

🧑‍🏫 **강사 해설**

README 요약은 몇 초 만에 먼저 옵니다. 정답은 API 5개입니다. 책 검색(`GET /api/books?q=`), 회원가입(`POST /api/signup`), 로그인 후 토큰 발급(`POST /api/login`), 토큰이 필요한 대출(`POST /api/books/:id/borrow`), 연체료가 함께 오는 반납(`POST /api/books/:id/return`)입니다.

test-runner 결과는 나중에 따로 도착합니다. 정답은 "테스트 2개 통과, 테스트가 없는 함수는 `returnBook`, `lateFee`, `signup`, `login`, `verify`"입니다. 지금 테스트는 `searchBooks` 와 `borrowBook` 만 부릅니다. `loadBooks`, `saveBooks` 는 그 안에서 간접적으로만 쓰입니다.

학생들은 이 단계에서 "동시에 하는 게 아니라 하나 끝나고 하나 하는 것 같다"고 자주 말합니다. 실제로는 동시에 돈 것이고, README 가 파일 하나만 읽으면 끝나는 짧은 일이라 먼저 끝났을 뿐입니다. 화면에 증거가 있습니다. README 답이 먼저 나오고, 그 아래 `Waiting for 1 background agent to finish` (뒤에서 1명이 아직 일하는 중)가 뜬 뒤, `Message from @test-runner` 로 결과가 도착합니다. 순서대로 했다면 먼저 시킨 test-runner 보고가 먼저 왔어야 합니다. 화면에 `Resuming agent ...` 가 보이면 test-runner 를 새로 만든 게 아니라 1-3 에서 쓴 것을 다시 불러온 것입니다.

Ctrl + B 로 뒤로 보낼 수 있는 건 **심부름꾼**과 **터미널 명령** 두 가지뿐입니다. 메인 클로드가 직접 생각하고 답을 쓰는 중에는 Ctrl + B 가 듣지 않습니다. 메인 클로드는 한 명이라 한 번에 한 가지만 생각하기 때문입니다. 그래서 "생각이 필요한 일까지 통째로 뒤에 맡기려면 심부름꾼이 필요하다"가 이 단계의 결론입니다. 그리고 강사 질문의 답처럼, 동시에 일하는 것까지는 심부름꾼도 되지만 **서로 대화하며 일하는 건 팀만** 됩니다. 블록 2 로 넘어가는 이유가 이것입니다.

블록 2

팀 켜기 + 첫 팀 (30분)

목표 — 팀 기능을 켜고, 파일을 고치지 않는 기획 회의로 첫 팀을 굴려 봅니다. 팀원이 일하는 화면에 들어가 직접 말을 거는 연습을 해야 팀원이 엉뚱한 방향으로 갈 때 바로잡을 수 있습니다.

2-1. 팀 기능 켜기 (7분)

💬 **클로드 대화창**

```
이 프로젝트의 .claude/settings.json 에 env 로 CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 을 추가해줘.  # 팀 기능 스위치 켜기 (이 프로젝트에서만)
기존 설정이 있으면 합쳐줘. 저장 후 파일 전체를 보여줘.  # 기존 설정을 덮어쓰지 않게 + 결과 눈으로 확인
```

👩‍🏫 **강사 해설**

결과로 `.claude/settings.json` 에 아래 내용이 들어가면 정답입니다.

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

설치는 필요 없습니다. Claude Code 에 이미 들어 있는 기능을 스위치로 켜는 것뿐입니다.

반드시 껐다 켜야 하는 이유는 클로드가 **켜질 때** 설정 파일을 읽기 때문입니다. 껐다 켜지 않으면 이후 2-2 에서 팀 대신 심부름꾼이 생기고, 현황판에서 ↓ 를 눌러도 아무것도 안 뜨는 증상이 나옵니다. 오후까지 "팀이 안 만들어져요"라는 질문의 대부분이 이 재시작 누락입니다.

이 프로젝트에만 켜는 이유도 설명해 주세요. 이 스위치가 켜져 있으면 클로드가 스스로 이름을 붙여 보낸 심부름꾼이 팀원으로 뜨는 경우가 있어서, 팀을 원하지 않은 프로젝트에서도 사용량이 늘 수 있습니다.

2-2. 첫 팀: 예약 기능 기획 회의 (15분)

💬 **클로드 대화창**

```
이 도서 대출 앱에 "예약 기능"(재고가 0인 책을 예약해두면 반납될 때 순서대로 받을 수 있는 기능)을 추가하려고 해.  # 팀원은 이 대화를 모르니 배경을 프롬프트에 직접 적기
agent team 으로 팀원 3명을 뽑아줘. 이름은 ux, arch, critic, 모델은 모두 Sonnet.  # "agent team 으로" = 심부름꾼 말고 팀. 이름을 정해야 나중에 이름으로 부를 수 있음
- ux: 사용자 입장에서 화면 흐름과 안내 문구  # 관점 1: 사용자
- arch: 데이터 구조와 API 설계 (src/books.js, server.js 기준)  # 관점 2: 설계. 볼 파일까지 지정
- critic: 악마의 변호인. 이 기능의 문제점과 빠뜨린 경우를 찾기  # 관점 3: 일부러 반대하는 비평가
코드는 고치지 말고, 결과를 합쳐서 docs/reservation-plan.md 에 써줘.  # 기획만. 팀장이 세 결과를 한 문서로 합침
```

👩‍🏫 **강사 해설**

결과로 `docs/reservation-plan.md` 가 생깁니다. 실제로 돌려 본 결과에서 "팀이라서 생긴 것"을 학생에게 짚어 줄 부분은 세 가지입니다.

첫째, **의견마다 누가 냈는지 표시**되어 있습니다. "(arch 제안, critic 조건부 찬성)", "(critic E4)"처럼 출처가 붙습니다. 혼자 쓴 문서에는 이런 흔적이 없습니다.

둘째, **팀원끼리 반박해서 결론이 바뀐 흔적**이 남습니다. 실제 결과에서 arch 는 처음에 "남은 권수(`available`)를 파일에 저장하자"고 했고, critic 이 "그러면 파일을 손으로 고치거나 중간에 실패할 때 다시 어긋난다"고 반박해서, 최종안은 "남은 권수는 저장하지 않고 그때그때 계산한다"로 바뀌었습니다. 혼자 하면 처음 떠올린 안으로 그냥 가는 경우가 많습니다. 이게 팀을 쓰는 가장 큰 이유입니다.

셋째, **"이 기능이 정말 필요한가?"라는 질문**이 나옵니다. critic 은 알림 기능이 없는데 보관 기한을 두는 모순을 지적하고, 대안 네 가지를 비교한 뒤 "예약보다 재고 버그와 토큰 위조부터 고쳐라"고 권했습니다. 만드는 쪽 관점만 있으면 이런 질문은 잘 안 나옵니다.

기획이 첫 팀 과제로 좋은 이유는 관점끼리 독립적이고 파일을 고치지 않아 사고가 날 일이 없기 때문입니다. 단, 이 기획안의 판단은 대부분 코드를 **읽고** 내린 것이지 실행해서 확인한 것은 아니라는 점도 같이 말해 주세요. "확인"의 차이는 블록 3 에서 숫자로 드러납니다.

2-3. 현황판 조작 연습 (8분) — 팀이 일하는 동안

⌨️ **키 조작** (2-2 프롬프트를 보내자마자)

```
↓ (입력창을 비운 채)  # 입력창 아래 현황판으로 내려가기. ↑ 는 이전 입력(히스토리)이 나옴
↑ / ↓              # 팀원 고르기 (critic 에 맞추기)
Enter              # 그 팀원 화면(대화창) 열기 → 여기서 입력하면 그 팀원에게 바로 전달
Esc                # 팀장 화면으로 돌아오기. 팀원 화면에서 누르면 그 팀원 작업이 멈춤 → "계속해"
Ctrl + T           # 팀 전체 할 일 보드 (대기 / 진행 중 / 완료)
```

💬 **critic 화면에서**

```
초등학생이 쓴다고 생각하고 문제점을 하나 더 찾아봐.  # critic 화면 안에서 입력 → 팀장을 거치지 않고 critic 에게 바로 전달
```

💬 **팀장 화면에서**

```
팀원 모두 퇴근시켜줘.  # 끝난 팀원 종료 (살아 있으면 사용량을 계속 씀)
```

👩‍🏫 **강사 해설**

이 단계에서 가장 많이 막히는 곳은 "↓ 를 눌러도 아무것도 안 뜬다"입니다. 원인은 거의 항상 **팀이 이미 일을 끝냈기 때문**입니다. 기획 팀은 1~2분 만에 끝나고, 모든 팀원이 쉬면 30초 뒤 현황판에서 숨겨집니다. 이때 학생이 입력창에 "초등학생이 쓴다고 생각하고..."를 그냥 치면, critic 이 아니라 **팀장**이 그 말을 받아서 직접 처리합니다. 실제 수업에서도 화면에 `Read 1 file`, `Update(docs/reservation-plan.md)` 가 떴는데, 이건 팀장이 한 일이었습니다. 그래서 팀장에게 "critic 팀원에게 메시지를 보내줘"라고 이름으로 부탁해야 critic 이 깨어납니다. 깨어나면 줄이 다시 생기니 그때 ↓ → Enter 로 들어가면 됩니다.

↓ 를 눌러도 안 뜨고, 팀장에게 이름으로 보내도 "그런 팀원이 없다"는 답이 오면 팀이 처음부터 안 만들어진 겁니다. 2-1 의 설정과 재시작, 2-2 프롬프트의 "agent team 으로"를 확인하게 하세요.

critic 이 찾는 추가 문제의 예(실제 결과)는 이렇습니다. 교실 공용 PC 에서는 로그아웃 버튼이 없고 로그인 토큰이 기한 없이 남아서, 뒷사람이 자기도 모르게 앞사람 계정으로 예약하거나 취소할 수 있다는 지적이었습니다. 여기에 "보관", "수령", "기한" 같은 단어가 초등학생에게 어렵다는 점, 금요일에 반납된 책은 3일 보관 기한이 주말에 끝나 버린다는 점까지 나왔습니다. 기획안에 "공용 PC 대응" 절이 새로 붙으면 정답입니다.

팀원 화면에 직접 말을 거는 게 좋은 이유는, 그 팀원이 지금까지 읽은 파일과 생각을 그대로 가진 채 방향만 바꿀 수 있기 때문입니다. 팀장을 거치면 팀장이 말을 요약해서 전달하기 때문에 뜻이 조금씩 바뀌고, 한 번 더 기다려야 합니다.

☕ 휴식 (10분)

⌨️ **터미널** (클로드 대화 중이면 새 터미널을 하나 더 열어서)

```
git checkout data/books.json                  # 혹시 바뀐 책 데이터가 있으면 처음 상태로 (0-2 참고)
git add . && git commit -m "docs: 예약 기능 기획"  # 지금까지 만든 파일(역할 카드, 설정, 기획 문서)을 담아서(add) 세이브(commit)
```

블록 3

🔍 리뷰 팀 — 혼자 vs 팀 대결 (50분)

목표 — 같은 코드를 혼자 리뷰한 결과와 팀으로 리뷰한 결과를 숫자로 비교합니다. 직접 비교해 봐야 "팀이 언제 이득이고 언제 낭비인지"에 대한 근거가 생깁니다. 오늘의 핵심 블록입니다.

3-1. 🧑 혼자 리뷰 (10분) — 기준점

⌨️ **새 터미널**을 하나 더 열고

```
cd ~/projects/library-app && claude   # 앱 폴더로 이동한 뒤 새 클로드 대화 시작
```

💬 **클로드 대화창**

```
이 앱의 문제점을 리뷰해줘. 보안, 성능, 버그, 테스트 부족 무엇이든.   # 비교 기준용 혼자 리뷰 (새 터미널 = 깨끗한 클로드)
서브에이전트나 팀은 쓰지 말고 너 혼자 해줘. 코드는 고치지 말고,   # 공정한 비교를 위해 혼자로 못 박기
문제마다 한 줄씩 번호를 매겨서 docs/review-solo.md 에 써줘.   # 번호를 매겨야 개수를 셀 수 있음
```

→ 몇 개 찾았는지 **칠판에** 적고 💬 `/exit` 로 이 터미널은 닫기

> 👩‍🏫 **강사 해설**
>
> 실제로 돌려 본 혼자 리뷰는 **44개**를 찾았습니다(보안 12, 버그·안정성 18, 성능 4, 테스트 부족 8, 기타 2). 심어 둔 13개는 모두 그 안에 들어 있었습니다. 반마다 숫자는 다르지만 "혼자서도 꽤 많이, 거의 다 찾는다"는 결과는 비슷하게 나옵니다.
>
> 혼자 리뷰의 특징은 두 가지입니다. 첫째, **범위가 넓습니다.** 한 머릿속이 모든 파일을 다 보기 때문에 "가입이 실패해도 항상 '가입 완료'가 뜬다", "새로고침하면 로그인 표시가 사라진다", "포트가 이미 쓰이고 있을 때 오류 처리가 없다" 같은 화면·운영 문제까지 잡습니다. 둘째, **직접 실행해서 확인하지 않습니다.** 코드를 읽고 "이렇게 될 것이다"라고 씁니다. 그래서 틀린 주장이 섞입니다. 실제 결과에서 "`__proto__` 라는 아이디로 가입 없이 로그인된다"는 주장이 있었는데, 블록 3-3 의 팀이 직접 해 보니 401 로 막혔습니다. 구조상 위험한 건 맞지만 말한 대로 뚫리지는 않은, **거짓 경보**였습니다.
>
> 이 두 특징을 칠판에 적어 두면 3-4 에서 비교할 때 바로 쓸 수 있습니다.

3-2. 역할 카드 3장 (10분)

💬 **원래 터미널의 클로드 대화창** (팀 기능이 켜진 쪽)

```
.claude/agents/ 에 팀원용 역할 카드 3개를 만들어줘: security-reviewer, perf-reviewer, test-reviewer.   # 관점별 전문가 카드 3장
- 모두 tools 는 Read, Glob, Grep, Bash (Bash 는 재현용으로만, 파일 수정 금지)   # Bash = 직접 실행해서 증명(재현)하라고 주는 도구
- model: sonnet   # 빠르고 비용이 적당한 모델
- 결과는 표로: 번호 ¦ 심각도(높음/중간/낮음) ¦ 파일:줄 ¦ 문제 ¦ 재현 방법 ¦ 고치는 방법   # 세 팀원 결과를 합치기 쉽게 표 모양 통일
- 추측이면 "추측" 이라고 표시   # 확인한 것과 짐작한 것을 구분
각 카드의 description 은 그 역할이 언제 쓰이는지 분명하게.   # 클로드가 카드를 고를 기준
```

> 👩‍🏫 **강사 해설**
>
> 결과로 `security-reviewer.md`, `perf-reviewer.md`, `test-reviewer.md` 세 파일이 생기고, 블록 1 의 `test-runner.md` 까지 합쳐 4개가 보이면 정답입니다. test-runner 는 이 리뷰 팀에는 쓰지 않고, 블록 5-4 에서 고친 결과를 검사할 때 다시 씁니다.
>
> 세 카드 모두 Bash 를 넣는 이유를 꼭 설명해 주세요. 혼자 리뷰와 팀 리뷰의 결과 차이 대부분이 여기서 나옵니다. Bash 가 있어야 팀원이 서버를 띄우고 실제로 공격 요청을 보내서 "진짜로 일어난다"를 증명할 수 있습니다. 결과 표에 "재현 방법"과 "추측" 표시를 요구한 것도 같은 이유입니다. 확인한 것과 짐작한 것을 구분해야 나중에 믿고 고칠 수 있습니다.

3-3. 🏢 팀 리뷰 (20분)

💬 **클로드 대화창**

```
agent team 으로 리뷰 팀을 꾸려줘. 팀원 3명:   # 같은 리뷰를 이번엔 팀으로
- sec: security-reviewer 에이전트 타입. src/auth.js, server.js, public/index.html 의 보안   # 에이전트 타입 = 3-2 의 역할 카드를 입혀서 뽑기
- perf: perf-reviewer 에이전트 타입. src/books.js, server.js 의 성능·안정성(서버가 죽을 수 있는 경우 포함)   # 파일 범위를 나눠 중복 조사 방지
- test: test-reviewer 에이전트 타입. src/books.js 의 로직 버그와 test/ 의 빠진 테스트   # 범위를 좁히면 깊어지지만 경계에 빈틈이 생김
코드는 고치지 마. 끝나면 결과를 합쳐서 docs/review-team.md 에 써줘. 같은 문제는 하나로 합치고 심각도 순으로 정렬해.   # 팀장이 합치기·중복 제거·정렬
```

⌨️ **팀이 일하는 동안**

```
Ctrl + T              # 할 일 보드: 누가 무엇을 하는 중인지
↓ → sec 고르고 Enter     # sec 화면 열기
재현해본 것만 '확인됨' 으로 표시해줘   # sec 화면에서 직접 지시 (팀장을 거치지 않음)
```

> 👩‍🏫 **강사 해설**
>
> 실제로 돌려 본 팀 리뷰는 sec 11건, perf 11건, test 9건, 모두 31건을 냈고, 팀장이 같은 문제를 합쳐 **29개**로 정리했습니다(높음 13, 중간 9, 낮음 5, 범위 밖 2). 혼자 리뷰의 44개보다 적습니다.
>
> 대신 혼자 리뷰에 없던 것이 세 가지 있습니다. 첫째, **확인 열**입니다. 항목마다 "재현(실제로 실행해서 확인)", "코드(읽고 확인)", "추측"이 표시됩니다. 실제로 팀원들은 `victim` 을 base64 로 바꾼 가짜 토큰으로 로그인 없이 대출에 성공했고, 숫자 `0` 으로 가입한 뒤 문자열 `"0"` 으로 로그인했고, `//` 주소 하나로 서버가 죽는 것을 확인했고, 책 10만 권을 넣고 대출 한 번에 0.11초가 걸리는 것을 쟀습니다. 혼자 리뷰는 같은 문제를 "느려진다", "위조할 수 있다"고만 썼습니다. 둘째, **출처 열**입니다. 경로 탈출처럼 sec 와 perf 가 따로따로 같은 문제를 찾은 항목은, 두 사람이 독립적으로 찾았으니 그만큼 믿을 만합니다. 셋째, 혼자 리뷰의 `__proto__` 주장을 실제로 해 보고 **"401 로 막혀서 실제 악용은 확인되지 않음"**으로 낮췄습니다.
>
> 반대로 팀이 놓친 것도 있습니다. 문서 끝의 "팀이 범위 밖으로 남긴 항목"이 그것입니다. test 팀원이 `src/books.js` 만 맡았기 때문에 `auth.js` 테스트와 서버 전체 테스트가 없다는 건 아무도 맡지 않았고, test 팀원 스스로 "범위 밖"이라고 밝혔습니다. 팀장이 혼자 리뷰와 비교해서 채워 넣었습니다. 화면 동작 버그("가입 완료"가 항상 뜨는 것 등)도 sec 가 `index.html` 을 보안 관점으로만 봤기 때문에 빠졌습니다. **일을 나누면 깊어지지만 경계에 빈틈이 생긴다**는 점이 이 결과의 교훈입니다.
>
> sec 화면에 들어가 "재현해본 것만 '확인됨'으로 표시해줘"라고 시키는 연습은, 팀원에게 직접 기준을 바꿔 줄 수 있다는 걸 보여주는 용도입니다. 표의 확인 열이 이 지시로 생기기도 합니다.

3-4. 🏆 점수판 (10분)

💬 **클로드 대화창** (혼자 리뷰 터미널과 팀 리뷰 터미널 각각)

```
/usage   # 지금까지 쓴 사용량 보기 → 혼자 vs 팀 사용량을 칠판에
```

→ `docs/review-solo.md` 와 `docs/review-team.md` 의 개수를 비교하고, 아래 정답지로 채점

> 👩‍🏫 **강사 해설**
>
> 🔒 **정답지: 연습용 앱에 심어 둔 문제 13개**
>
> 학생 리뷰가 아래와 다른 **진짜 문제**를 찾으면 가산점을 줍니다.
>
> **보안 (S)**
>
#	파일	문제	재현
S1	`src/auth.js`	비밀번호를 **평문**으로 저장	코드 확인
S2	`src/auth.js` `signup`	같은 아이디로 가입하면 **기존 계정 비밀번호를 덮어씀** (계정 탈취)	`hong` 으로 두 번 가입 → 두 번째 비밀번호로 로그인됨
S3	`src/auth.js` `login`	`users[username] == password` → **없는 아이디 + 비밀번호 생략** 시 `undefined == undefined` 로 로그인 성공	`curl -XPOST localhost:3000/api/login -d '{"username":"nobody"}'` → 토큰 발급
S4	`src/auth.js` `verify`	토큰 = 아이디의 base64. 서명이 없어 **누구나 위조**	`Authorization: $(echo -n hacker | base64)` 로 대출 성공
S5	`server.js` 정적 파일	`path.join('public', req.url)` → **경로 탈출**로 서버 소스 노출	`curl --path-as-is http://localhost:3000/../server.js`
S6	`public/index.html`	책 제목·저자를 `innerHTML` 로 출력 → **XSS** 위험 (데이터에 HTML 이 들어오면 실행)	코드 확인
>
> **성능·안정성 (P)**
>
#	파일	문제	재현
P1	`server.js` `readBody`	`JSON.parse` 에 try/catch 없음 → **잘못된 JSON 한 번에 서버 전체 다운**	`curl -XPOST localhost:3000/api/login -d '{bad'` → 서버 종료
P2	`src/books.js`	요청마다 `readFileSync` / `writeFileSync` 로 파일 전체를 **동기** 읽기·쓰기	코드 확인
P3	`src/books.js` `searchBooks`	이중 반복문 + 여러 단어가 맞으면 **같은 책 중복** 반환	`/api/books?q=자바스크립트 가이드` → 3번 책이 2번
P4	`src/books.js` `loans`	대출 기록이 **메모리에만** → 서버 재시작하면 반납 불가	대출 → 서버 재시작 → 반납
>
> **로직·테스트 (L, T)**
>
#	파일	문제	재현
L1	`src/books.js` `borrowBook`	`available >= 0` → **재고 0인데 대출**되어 -1 까지 내려감	2번 책(1권) 두 번 대출 → `available: -1`
L2	`src/books.js` `lateFee`	일찍 반납하면 **음수 연체료** (2일 만에 반납 → -1200원)	`lateFee(1월 1일, 1월 3일)`
T1	`test/`	정상 경로 테스트 2개뿐. 재고 0 대출, 반납, 연체료, 인증 테스트 **없음**	`npm test` 결과 확인
>
> **채점 기준**: 13개 중 몇 개를 찾았나(1개당 1점) + 실제로 실행해서 재현까지 한 것은 ×2. 결과는 반마다 다르니 **혼자 / 팀 평균을 기록**해두면 다음 수업에서 근거로 쓸 수 있습니다.
>
> **실제로 돌려 본 결과 (채점 예시)**
>
#	혼자	팀
S1 평문 저장	✓	✓ 코드
S2 가입 덮어쓰기	✓	✓ **재현**
S3 `undefined == undefined`	✓	✓ **재현**
S4 토큰 위조	✓	✓ **재현**
S5 경로 탈출	✓	✓ **재현**
S6 XSS	✓	✓ 추측
P1 잘못된 JSON 서버 다운	✓	✓ **재현**
P2 동기 파일 읽기·쓰기	✓	✓ **재현(측정)**
P3 검색 중복	✓	✓ **재현**
P4 대출 기록 메모리	✓	✓ 코드
L1 재고 0 대출	✓	✓ **재현**
L2 음수 연체료	✓	✓ **재현**
T1 테스트 부족	✓	✓ **재현**
찾은 개수	**13 / 13**	**13 / 13**
점수 (재현 ×2)	**13점**	**23점**
문제 목록 전체	44개	29개
거짓 경보	`__proto__` 로그인 1건	실행해 보고 걸러 냄
>
> 결과를 문장으로 정리하면 이렇습니다.
>
> **찾은 개수는 비겼습니다.** 혼자도 팀도 심어 둔 13개를 다 찾았고, 문제 목록 전체로는 오히려 혼자가 44개로 더 많았습니다. 파일 7개짜리 작은 앱은 클로드 한 명의 머릿속에 다 들어가기 때문에, 혼자서도 넓게 볼 수 있습니다.
>
> **믿을 수 있는 정도는 팀이 이겼습니다.** 점수 차이 10점은 전부 "직접 실행해서 확인했나"에서 났습니다. 팀원은 각자 맡은 파일만 깊게 보니까 서버를 띄우고 공격 요청을 보내 볼 여유가 생겼고, 그 과정에서 혼자 리뷰의 거짓 경보(`__proto__`)도 걸러졌습니다. 혼자 리뷰를 그대로 믿고 고쳤다면, 실제로는 안 뚫리는 곳을 고치느라 시간을 썼을 겁니다.
>
> **범위는 혼자가 이겼습니다.** 팀은 일을 나눈 탓에 아무도 맡지 않은 경계(auth 테스트, 화면 동작 버그)를 놓쳤고, 팀장이 메웠습니다. 팀을 쓸 때는 "이 범위 나누기에서 빠진 곳이 없나"를 팀장이나 내가 꼭 확인해야 합니다.
>
> **사용량은 팀이 몇 배였습니다.** `/usage` 로 혼자 리뷰를 한 터미널과 팀 리뷰를 한 터미널의 사용량을 비교하면, 팀원 3명에 팀장까지 돌아가므로 보통 몇 배가 나옵니다.
>
> 그래서 강사 질문의 답은 이렇게 정리됩니다. "팀이 더 **많이** 찾지는 않았다. 더 **확실하게** 찾았다. 그 대신 몇 배를 썼다. 그러니 매번 팀을 쓰지는 않는다. 이 정도 작은 앱의 1차 점검은 혼자로 충분하고, 배포 직전처럼 '진짜로 일어나는지'가 중요하거나, 코드가 커서 한 명이 다 못 보는 경우에 팀을 쓴다."
>
> 학생들에게 이렇게 물어보면 결론을 스스로 말하게 할 수 있습니다. "내일 이 앱을 실제로 배포한다면 어느 리뷰를 믿겠어요?" 그리고 "팀이 놓친 화면 버그는 범위를 어떻게 나눴으면 잡았을까요?" 두 번째 질문의 답은 "화면 동작 담당 팀원을 하나 더 두거나, `index.html` 의 동작 버그를 누군가에게 명시적으로 맡긴다"입니다.

블록 4

💬 토론 + 직접 대화 (40분)

목표 — 팀원끼리 서로의 리뷰 결과를 반박하게 해서 거짓 경보를 걸러 내고, 오후에 고칠 순서와 담당자를 정합니다. 서로 반박하는 건 심부름꾼은 못 하고 팀만 할 수 있는 일입니다.

4-1. 반박 회의 (15분)

💬 클로드 대화창

```
리뷰 팀원 3명에게 전달해줘: docs/review-team.md 를 같이 읽고,   # 팀장을 통해 팀원 전원에게 지시
다른 팀원이 찾은 문제 중 과장됐거나 틀렸다고 생각하는 것을 그 팀원에게 직접 반박해.   # 팀원끼리 메신저로 직접 따지기 (팀만 가능)
반박당한 팀원은 재현으로 증명하거나 철회해. 합의된 결과로 review-team.md 를 고쳐줘.   # 증명 못 하면 철회 → 거짓 경보 걸러내기
```

🧑‍🏫 강사 해설

반박의 대상은 주로 "추측"이나 "코드"로 표시된 항목입니다. 3-3 의 실제 결과를 기준으로 하면 반박이 붙기 쉬운 항목은 이렇습니다. 큰 요청 본문으로 메모리를 고갈시킨다는 항목은, 서버가 죽은 이유가 메모리 때문인지 JSON 파싱 실패 때문인지 구분하지 못했다고 스스로 적었습니다. XSS 는 코드로만 확인했습니다. 문자열 id `'1'` 로는 책을 못 찾는다는 항목과 `lateFee` 가 `NaN` 을 낸다는 항목은 실행하지 않았습니다. 반박을 받은 팀원은 재현해서 증명하거나, 심각도를 낮추거나, 철회합니다. 결과적으로 review-team.md 의 "추측"이 줄고 "재현"이 늘면 정답입니다. 몇 개가 바뀌는지는 반마다 다릅니다.

이 단계가 보여주는 건 **팀만 할 수 있는 일**입니다. 심부름꾼은 보고하고 사라지기 때문에 다른 심부름꾼의 결과를 볼 수 없고, 서로 반박할 수도 없습니다. 팀원은 메신저로 서로 이름을 불러 직접 따지고, 팀장은 합의된 결과만 정리합니다. 리뷰에 거짓 경보가 섞이는 건 피할 수 없으니, 그걸 걸러 내는 장치가 있다는 게 팀의 값어치입니다. 혼자 리뷰에서 나온 `__proto__` 같은 거짓 경보가 여기서 걸러지는 대표적인 예입니다.

4-2. 내가 직접 끼어들기 (10분)

```
"서버가 죽는 문제"가 있다면 그게 제일 급해. 재현 명령어를 정확히 알려줘.   # perf 화면 안에서 직접 질문
```

🧑‍🏫 강사 해설

perf 가 돌려줄 정답은 서버를 죽이는 요청들입니다. 실제 결과에서 나온 것은 다음과 같습니다.

```
curl -XPOST localhost:3000/api/login -d '{bad'   # 잘못된 JSON → JSON.parse 예외로 서버 종료 (P1)
curl -XPOST localhost:3000/api/login -d '{}'     # 아이디 없이 로그인 → Buffer.from(undefined) 예외로 서버 종료
curl --path-as-is localhost:3000//               # 이상한 주소 → new URL 예외로 서버 종료
```

`data/books.json` 이 깨져 있으면 검색 한 번에 서버가 죽는다는 것도 함께 나옵니다. 셋 다 로그인 없이 요청 한 번이면 되기 때문에, 고칠 순서에서 맨 앞에 와야 합니다.

팀원에게 직접 묻는 이유는, perf 가 자기가 조사하면서 본 것을 전부 기억한 채로 답하기 때문입니다. 팀장에게 물으면 팀장은 perf 의 요약만 가지고 있어서 다시 perf 에게 물어봐야 하고, 그 과정에서 명령어가 빠지거나 바뀝니다.

4-3. 수리 계획 (10분)

💬 클로드 대화창 (팀장 화면에서)

```
review-team.md 를 바탕으로 docs/fix-plan.md 를 만들어줘.   # 리뷰 결과 → 고칠 계획
- 심각도 높음부터   # 급한 것부터
- 각 문제를 고칠 파일 (src/books.js / src/auth.js / public/index.html / server.js) 열 추가   # 파일 기준으로 나눠야 오후에 겹치지 않음
- 오후에 팀원 3명(books, auth, web)이 나눠 고칠 거니까 담당자 열도 추가. server.js 는 팀장 담당   # 공통 파일 server.js 는 한 명만
```

🧑‍🏫 강사 해설

`docs/fix-plan.md` 의 담당자 배정은 대략 아래처럼 나오면 정답입니다.

담당	파일	고칠 문제
books	`src/books.js`	L1 재고 0 대출, L2 음수 연체료, P3 검색 중복, P4 대출 기록 저장 (P2 동기 읽기·쓰기는 선택)
auth	`src/auth.js`	S1 비밀번호 해시, S2 중복 가입 거부, S3 비밀번호 비교, S4 서명 토큰
web	`public/index.html`	S6 XSS (`textContent` 로 바꾸기), 검색어 인코딩, 가입 실패 처리
팀장	`server.js`	S5 경로 탈출, P1 잘못된 JSON 처리, 전체 예외 처리, 본문 크기 제한

이 표를 오전에 만들어 두는 이유는, 오후 구현 팀에서 가장 큰 사고가 **두 팀원이 같은 파일을 동시에 고쳐 서로 덮어쓰는 것**이기 때문입니다. 문제를 파일 기준으로 나누면 팀원끼리 파일이 겹치지 않습니다. `server.js` 는 모든 기능의 주소가 모이는 공통 파일이라 누구 한 명에게 주면 다른 팀원이 기다려야 하므로, 팀원들이 끝난 뒤 팀장이 한 번에 합치게 합니다.

4-4. 퇴근 + 저장 (5분)

💬 클로드 대화창

```
팀원 모두 퇴근시켜줘.   # 리뷰 팀 종료
```

⌨️ 터미널

```
git add . && git commit -m "docs: 리뷰 결과와 수리 계획"   # 리뷰 문서와 수리 계획을 세이브
```

퀴즈

🧩 오전 복습 퀴즈 (20분)

목표 — 오전에 한 것을 상황 문제로 다시 떠올려서, 혼자·심부름꾼·팀을 고르는 기준을 굳힙니다.

→ 상황을 하나씩 읽어 주고 팀 대항으로 "혼자 / 심부름꾼 / 팀" 을 고르게 한 뒤, 증상을 읽어 주고 원인을 맞히게 합니다.

👩‍🏫 강사 해설

퀴즈 1. "혼자? 심부름꾼? 팀?" 정답

상황	정답	이유
오타 하나 고치기	🧑 혼자	너무 작음
로그 5만 줄에서 에러 원인 찾기	🏃 심부름꾼	결과만 필요, 긴 로그가 메인 대화에 안 쌓이게
PR 하나를 보안·성능·테스트로 동시에 리뷰	🏢 팀	관점이 여러 개
A 끝나야 B, B 끝나야 C 인 작업	🧑 혼자	순서가 있으면 팀원이 서로 기다리기만 함
원인 모르는 버그, 가설이 여러 개	🏢 팀	서로 반박하는 토론이 필요
같은 파일 하나를 크게 고치기	🧑 혼자	여럿이 같은 파일 = 덮어쓰기 사고
프론트·백엔드·테스트를 다른 파일에서 개발	🏢 팀	파일이 안 겹치게 나눌 수 있음
테스트만 돌리고 실패 요약	🏃 심부름꾼	상의할 필요 없음

"PR 하나를 보안·성능·테스트로 동시에 리뷰"의 정답이 팀인 것은 맞지만, 블록 3 의 결과를 같이 짚어 주세요. 팀이 이긴 건 찾은 **개수**가 아니라 **확인의 깊이**였습니다. 그러니 정확한 답은 "PR 이 크거나, 진짜로 일어나는지 확인이 중요하면 팀, 작고 1차 점검이면 혼자"입니다. 퀴즈 답을 외우는 게 아니라 "왜?"에 오늘 본 숫자로 답하게 하는 게 목적입니다.

퀴즈 2. "왜 이렇게 됐지?" 정답

증상	원인
팀원이 내가 아까 설명한 배경을 모름	팀원은 **팀장과 나눈 대화를 모름** → 지시에 직접 적기
팀원 줄이 현황판에서 사라짐	**30초 뒤 숨김** (죽은 게 아님)
팀 대신 심부름꾼이 생김	"**agent team 으로**" 를 안 적음
두 팀원이 고친 내용 중 하나가 사라짐	**같은 파일**을 둘이 고침 → 파일 주인 정하기
팀원 화면에서 `/model` 이 안 먹힘	팀원 모델은 **뽑을 때 고정**
사용량이 갑자기 확 줄었음	팀원 수만큼 사용 → 끝난 팀원 퇴근

블록 5

🛠️ 구현 팀 — 수리 + 예약 기능 (60분)

목표 — 팀원 여럿이 동시에 코드를 고칩니다. 같은 파일을 둘이 고치면 서로 덮어쓰기 때문에, 세이브(커밋)·파일 주인·계획 먼저 확인이라는 안전장치를 먼저 걸고 시작합니다.

5-1. 안전장치 (10분)

⌨️ **터미널**

```
git status   # "nothing to commit, working tree clean" 이 나와야 함. 아니면 git add . && git commit 먼저
```

💬 **클로드 대화창**

```
.claude/settings.json 의 permissions.allow 에 "Bash(npm test:*)", "Bash(node --test:*)" 를 추가해줘. 기존 설정은 유지해줘.   # 안전한 테스트 명령은 묻지 말고 실행 → 허락 요청 폭탄 방지
```

> 👩‍🏫 **강사 해설**
>
> `git status` 가 `nothing to commit, working tree clean` 이면 정답입니다. `data/books.json` 이 수정됨(`M`)으로 보이면 `git checkout data/books.json` 으로 되돌리고, 그 외 파일이 보이면 커밋부터 합니다. 이 커밋이 "팀이 망쳤을 때 돌아갈 곳"입니다. 팀원 3명이 동시에 고치기 시작하면 무엇이 누구 때문에 바뀌었는지 사람이 추적하기 어렵기 때문에, 시작점을 깨끗하게 찍어 두는 겁니다.
>
> 허락 목록을 넣지 않으면, 팀원들의 "이 명령 실행해도 돼요?" 요청이 전부 팀장 화면으로 올라와서 학생이 계속 Enter 만 누르게 됩니다. `npm test` 는 파일을 바꾸지 않는 안전한 명령이라 미리 허락해도 됩니다.

5-2. 파일 소유표 + 할 일 보드 (10분)

💬 **클로드 대화창**: **Shift + Tab** 을 눌러 `plan mode` 로 바꾼 뒤

```
docs/fix-plan.md 와 docs/reservation-plan.md 를 바탕으로 구현 팀 계획을 세워줘.   # Shift + Tab 으로 plan mode 에서 보냄 (읽기·계획만)
팀원 3명(books, auth, web), 모델은 Sonnet. 파일 소유표:   # 파일 주인 정하기
- books: src/books.js, test/books.test.js   # books 는 이 두 파일만
- auth: src/auth.js, test/auth.test.js (새로 만들기)   # auth 는 인증 코드와 새 테스트
- web: public/index.html   # web 은 화면만
- 팀장: server.js (API 라우트 추가·수정은 팀장만)   # 모두가 쓰는 공통 파일은 팀장만
할 일 보드는 팀원 1명당 4~6개, 선행 작업 관계도 넣어줘 (예: 예약 API 는 books 의 예약 로직이 끝난 뒤).   # 순서가 있는 일은 잠가 두기
각 팀원 업무 지시서(역할/범위/배경/할 일/결과물)를 먼저 보여주고, 내가 OK 하면 그때 팀원을 뽑아.   # 뽑기 전에 내가 먼저 확인 (팀원은 이 대화를 모름)
```

→ 지시서를 읽고 **파일 범위가 서로 겹치지 않으면** 다음 단계로

> 👩‍🏫 **강사 해설**
>
> 계획 모드에서 팀장은 팀원별 업무 지시서와 할 일 보드 초안을 보여주고, 팀원은 아직 뽑지 않습니다. 할 일 보드에는 "books: 예약 로직 → 팀장: 예약 API 주소 추가 → web: 예약 버튼"처럼 **선행 작업 관계**가 보이면 정답입니다.
>
> 학생이 지시서에서 확인할 것은 두 가지입니다. 첫째, **파일 범위가 겹치지 않는지**입니다. 예를 들어 books 지시서에 `server.js` 가 들어가 있으면 고쳐 달라고 해야 합니다. 둘째, **배경이 들어 있는지**입니다. 팀원은 오전에 팀장과 나눈 대화를 전혀 모릅니다. 지시서에 "review-team.md 의 몇 번을 고쳐라", "reservation-plan.md 의 데이터 구조를 따라라" 같은 배경이 없으면 팀원은 처음부터 다시 조사하거나 엉뚱한 방향으로 갑니다.
>
> 이 단계가 "절대 자르면 안 되는 것"인 이유는, 팀원을 뽑고 나서 방향이 틀린 걸 알면 이미 코드가 바뀌기 시작한 뒤라 되돌리기 어렵고 사용량도 이미 쓴 뒤이기 때문입니다. **팀은 시작 전에 확인하는 게 가장 싸다**가 결론입니다.

5-3. 팀 가동 (30분)

💬 **클로드 대화창**: **Shift + Tab** 으로 계획 모드를 끄고 (일반 모드로)

```
OK. 팀원을 뽑아서 진행해줘. 팀원들은 각자 할 일이 끝날 때마다 npm test 를 돌리고,   # Shift + Tab 으로 plan mode 끄고 보냄. 고칠 때마다 테스트
자기 파일만 git add 해서 "팀원이름: 한 일" 형식으로 커밋해.   # git add . 금지 → 남의 작업 중간 상태가 섞이지 않게
너는 팀원들이 끝날 때까지 기다리고 그동안 직접 구현하지 마. 팀원이 모두 끝나면 server.js 만 네가 고쳐.   # 팀장이 먼저 손대면 같은 파일 충돌
```

⌨️ **팀이 일하는 동안**

```
Ctrl + T                    # 선행 작업이 끝나면 잠긴 일이 풀리는지 관찰
팀원들이 끝날 때까지 기다려. 직접 구현하지 마   # 팀장이 직접 코드를 쓰기 시작하면
지금 어디까지 했어? 막힌 게 있어?   # 오래 멈춘 팀원 화면에 들어가서
```

> 👩‍🏫 **강사 해설**
>
> 진행되는 모습은 이렇습니다. Ctrl + T 할 일 보드에서 books 의 예약 로직이 완료되면 잠겨 있던 "예약 API" 와 "예약 버튼" 이 풀립니다. `git log --oneline` 에는 `books: ...`, `auth: ...`, `web: ...` 처럼 팀원 이름이 붙은 커밋이 쌓이고, 마지막에 팀장의 `server.js` 커밋이 붙습니다.
>
> 흔히 생기는 사고는 셋입니다. 첫째, 팀장이 기다리지 않고 직접 `books.js` 를 고치기 시작합니다. 그러면 books 팀원과 같은 파일을 동시에 고치게 되므로 바로 멈추게 합니다. 둘째, 팀원이 `git add .` 로 남의 파일까지 커밋해서, 다른 팀원이 고치던 중간 상태가 섞여 들어갑니다. 셋째, 팀원이 한 곳에서 오래 멈춥니다. 대부분 허락 요청을 기다리고 있거나, 선행 작업이 안 끝나서 잠긴 일을 기다리는 중입니다. "지금 팀장이 뭘 기다리고 있어요?"라는 질문이 여기서 쓰입니다.
>
> 구현 팀이 이득인 조건은 **파일이 겹치지 않게 나눌 수 있을 때**뿐입니다. 이 앱은 books, auth, 화면이 파일로 깔끔하게 나뉘어서 팀이 맞지만, 한 파일을 크게 고치는 일이었다면 혼자가 낫습니다(퀴즈 1 참고).

5-4. 확인 (10분)

⌨️ **터미널**

```
git log --oneline   # 팀원별 커밋("books: ...", "auth: ...")이 보이는지 확인
npm test            # 모든 테스트 통과 확인
npm start           # 서버 켜고 브라우저에서 예약 기능 눌러보기 → 끝나면 Ctrl + C
```

💬 **클로드 대화창**: 블록 1 에서 만든 test-runner 로 **다른 눈**으로 검사

```
@"test-runner (agent)" 테스트를 돌리고, 실패하면 원인과 파일:줄을 알려줘. 새로 생긴 테스트가 무엇을 검사하는지도 한 줄씩 정리해줘.   # 블록 1 의 test-runner 재사용 = 고친 사람과 다른 눈으로 검사
```

💬 **클로드 대화창**

```
팀원 모두 퇴근시키고, 할 일 보드에서 끝나지 않은 일이 있으면 알려줘.   # 퇴근 + 남은 일 점검
```

> 👩‍🏫 **강사 해설**
>
> `npm test` 결과는 오전의 `pass 2` 보다 테스트 수가 늘어 있어야 합니다. books 의 재고·연체료·검색 테스트와 auth 의 가입·로그인·토큰 테스트가 새로 생기기 때문입니다. 브라우저에서는 재고가 0 인 책에 [예약하기] 버튼이 보이면 됩니다.
>
> test-runner 를 여기서 다시 부르는 이유는 **고친 사람과 검사하는 사람을 나누기 위해서**입니다. 팀원들은 자기가 고친 코드라서 "다 됐다"고 믿기 쉽습니다. test-runner 는 이 코드를 고치는 데 참여하지 않았고, `tools` 에 Edit 가 없어서 고칠 수도 없기 때문에, 결과를 있는 그대로 보고합니다. 오전에 만든 역할 카드가 오후에 이렇게 다시 쓰인다는 점을 짚어 주세요. 역할 카드는 한 번 만들면 계속 꺼내 쓰는 도구입니다.
>
> 보너스의 `curl --path-as-is http://localhost:3000/../server.js` 는 고치기 전에는 서버 소스를 그대로 돌려줬지만, 팀장이 S5 를 고쳤다면 이제 403 이나 404 가 나와야 합니다.

블록 6

🚧 검문소 — 테스트 못 넘으면 완료 금지 (40분)

목표 — 팀원이 테스트를 깨뜨린 채 "완료"하는 것을 hook(검문소)으로 자동으로 막습니다. 팀원이 여럿이면 사람이 일일이 확인할 수 없어서, 말이 아니라 구조로 막아야 합니다.

6-1. 검문소 설치 (10분)

💬 **클로드 대화창**

```
.claude/settings.json 에 TaskCompleted hook 을 추가해줘.   # 할 일을 "완료"로 표시하려는 순간 자동 실행되는 검문소
command 는 "npm test 1>&2 ¦¦ exit 2" 로. 기존 env, permissions 설정은 유지해줘.   # 테스트 결과를 팀원에게 보여주고, 실패하면 exit 2 로 완료 거부
```

👩‍🏫 강사 해설

설정 파일에 아래 부분이 들어가면 정답입니다. 기존 `env`, `permissions` 는 그대로 남아 있어야 합니다.

```
"hooks": {
  "TaskCompleted": [
    { "hooks": [ { "type": "command", "command": "npm test 1>&2 ¦¦ exit 2" } ] }
  ]
}
```

뜻은 이렇습니다. 팀원이 할 일을 "완료"로 표시하려는 순간(`TaskCompleted`) Claude Code 가 `npm test` 를 대신 돌립니다. 테스트 결과는 `1>&2` 로 에러 출력에 실리고, 이 내용이 팀원에게 피드백으로 전달됩니다. 테스트가 실패하면 `¦¦` 뒤의 `exit 2` 가 실행되어 완료가 거부됩니다. 종료 코드가 1 이면 "실패했다"고만 기록되고 완료는 그대로 진행되므로, 반드시 2 여야 막힙니다.

6-2. 🔥 일부러 걸려보기 (20분)

💬 **클로드 대화창**

```
agent team 으로 팀원 1명(tester)을 뽑아줘, 모델은 Sonnet.   # 검문소 실험용 팀원 1명
할 일: test/books.test.js 에 "검색어 앞뒤에 공백이 있어도 결과는 같다" 테스트를 추가해   # 지금 코드로는 실패하는 테스트
(예: searchBooks(' 리팩터링 ') 의 결과가 1권). src/books.js 는 고치지 말고 테스트만 추가한 다음 할 일을 완료로 표시해.   # 일부러 실패한 채 완료 시도 → 검문소가 막는지 보기
```

```
tester 에게 전달해줘: 완료가 거부된 이유를 읽고, src/books.js 의 searchBooks 를 고쳐서 테스트를 통과시킨 뒤 다시 완료해.   # 거부 메시지를 읽고 고친 뒤 다시 완료 → 통과
```

👩‍🏫 강사 해설

지금 코드에서 `searchBooks(' 리팩터링 ')` 은 1권이 아니라 **11건**을 돌려줍니다. 앞뒤 공백 때문에 빈 단어 `''` 가 생기고, 빈 단어는 모든 제목에 들어 있는 것으로 판정되어 모든 책이 중복으로 나오기 때문입니다. 그래서 tester 가 쓴 테스트는 실패하고, tester 가 완료를 표시하려는 순간 검문소가 `npm test` 를 돌려 실패를 확인하고 `exit 2` 로 완료를 거부합니다. 테스트 실패 메시지는 `1>&2` 로 에러 출력에 실렸기 때문에 tester 에게 그대로 전달되고, tester 는 그걸 읽고 원인을 압니다. 두 번째 지시 후 tester 가 `split(/\s+/).filter(Boolean)` 처럼 빈 단어를 없애도록 고치면 테스트가 통과하고 완료가 받아들여집니다.

블록 5 에서 books 팀원이 검색을 이미 고쳤다면 이 테스트는 처음부터 통과해서 거부가 일어나지 않습니다. 그때는 아직 안 고친 다른 문제로 주제를 바꾸세요. 예를 들어 "기한 안에 반납하면 연체료는 0원이다"(L2)나 "재고 0인 책은 대출이 거부된다"(L1) 중 남은 것을 고르면 됩니다.

검문소가 필요한 이유는 1-2 에서 Edit 를 뺀 것과 같은 원리입니다. "테스트 통과시키고 완료해"라고 말로 시키면 팀원이 가끔 건너뜁니다. 검문소는 말이 아니라 **구조로** 막습니다. 팀원이 여럿일수록 사람이 일일이 확인할 수 없으니 이런 자동 장치의 가치가 커집니다. `exit 1` 이 아니라 `exit 2` 여야 하는 이유는, Claude Code 가 hook 의 종료 코드 2 만 "막아라"로 읽기 때문입니다.

6-3. 정리 (10분)

💬 **클로드 대화창**

```
팀원 퇴근시켜줘.   # tester 종료
```

⌨️ **터미널**

```
git add . && git commit -m "chore: TaskCompleted 검문소 + 검색 공백 테스트"   # 검문소 설정과 새 테스트를 세이브
```

블록 7

🕵️ 디버깅 토론 — 짝꿍이 심은 버그 찾기 (30분)

목표 — 원인을 모르는 버그를 팀원들의 경쟁 가설 토론으로 찾습니다. 클로드 한 명은 처음 떠올린 원인에 꽂히기 쉬워서, 서로 반박하게 해야 진짜 원인이 남습니다.

7-1. 버그 심기 (8분, 짝꿍과 자리 바꾸기)

```
cd ~/projects/library-app && claude   # 짝꿍 앱 폴더에서 새 클로드 대화 시작
```

💬 **클로드 대화창**

```
이 앱에 미묘한 버그를 하나만 몰래 심어줘. 조건:   # 짝꿍 컴퓨터에서 실행 (내가 심으면 답을 알아버림)
- 서버는 정상으로 켜지고, 화면에서 사용하다 보면 이상한 결과가 나오는 종류   # 바로 티 나지 않는 버그
- 주석이나 이름으로 티 내지 말 것   # 힌트 제거
- 바꾼 내용은 나에게만 말해줘. 그리고 git add . && git commit -m "chore: 정리" 로 커밋해.   # 커밋 메시지도 평범하게 위장
```

→ A 는 **증상 카드**만 종이에 적어서 B 에게 줌 (예: "반납했는데 남은 권수가 안 늘어나요") → 💬 `/exit` → 역할 바꿔서 B 도 A 의 컴퓨터에 버그 심기

7-2. 🔬 가설 토론 팀 (17분)

💬 **클로드 대화창** (내 컴퓨터, 팀 켜진 대화)

```
사용자 증상: (증상 카드 내용)   # 짝꿍이 준 증상 카드 내용으로 바꿔 넣기
agent team 으로 팀원 3명(모델 Sonnet)을 뽑아서 각자 다른 가설을 조사하게 해줘.   # 가설마다 담당자 1명 (경쟁 가설)
과학 토론처럼 서로의 가설을 반박하려고 노력하게 하고, git 기록은 보지 말고 코드와 실행으로만 증명해.   # 반박에서 살아남은 가설만 남김. git 기록은 답지라 금지
합의된 원인과 증거(재현 명령어)를 docs/findings.md 에 정리해줘. 아직 고치지는 마.   # 원인 확정 전에는 고치지 않기
```

→ 시간 재기 ⏱️ → 찾으면 짝꿍에게 확인 → 맞으면 🎉

> **👩‍🏫 강사 해설**
>
> 결과는 짝꿍이 심은 버그에 따라 다르지만, 진행 모양은 비슷합니다. 팀원 3명이 "books.js 의 계산 로직이 틀렸다", "server.js 에서 주소나 값을 잘못 넘긴다", "화면이 잘못된 값을 보여준다"처럼 서로 다른 가설을 맡고, 각자 재현 명령으로 자기 가설을 증명하거나 남의 가설을 깨려고 합니다. 반박에서 살아남은 가설 하나와 그걸 보여주는 재현 명령이 `docs/findings.md` 에 남으면 정답입니다.
>
> 이 블록이 팀에 유리한 이유는 **닻 내림 효과** 때문입니다. 클로드 한 명에게 시키면 처음 찾은 그럴듯한 원인에서 멈추는 경우가 많습니다. 한 번 "이게 원인이다"라고 생각하면 그 뒤 조사가 전부 그 생각을 확인하는 쪽으로 기울기 때문입니다. 팀원이 서로 다른 가설을 맡아 반박하면, 증거로 버틴 원인만 남습니다. 시간이 남으면 같은 증상을 혼자 클로드에게도 시켜 보고 걸린 시간과 맞혔는지를 비교하면, 블록 3 처럼 숫자로 근거를 남길 수 있습니다.
>
> git 기록을 금지한 이유는 `git log` 나 `git diff` 를 보면 짝꿍이 바꾼 줄이 바로 보여서 토론 없이 답이 나오기 때문입니다. 실제 현장에서는 git 기록이 가장 빠른 단서이므로 당연히 써야 한다는 점도 같이 말해 주세요. 오늘은 토론 연습이 목적이라 막은 것입니다.

7-3. 정리 (5분)

💬 **클로드 대화창**

```
findings.md 의 원인대로 고쳐줘. 그리고 같은 버그가 다시 생기지 않게 테스트도 하나 추가해줘.   # 고치기 + 재발 방지 테스트
```

⌨️ **터미널**

```
npm test && git add . && git commit -m "fix: 디버깅 토론으로 찾은 버그 수정"   # 테스트가 통과할 때만(&&) 담아서 세이브
```

블록 8

🚀 나만의 팀 — 2바퀴 (90분)

목표 — 강사 없이 학생이 직접 팀장이 되어 "이 일에 팀이 필요한가?"부터 판단하고 팀을 굴려 봅니다. 실제로 쓸 때는 이 판단이 가장 중요하기 때문입니다.

8-1. 고르기 + 팀 설계서 (20분)

→ 아래 메뉴에서 하나 고르고, 팀 설계서를 종이나 `docs/team-design.md` 에 써서 강사 확인 후 시작

난이도	상황	팀 구성 힌트
⭐	도서 앱 README·API 문서 리뷰	초보 사용자 / 개발자 / 깐깐한 편집자
⭐	기술 조사: "이 앱을 SQLite 로 옮기면?"	찬성 / 반대 / 옮기는 방법 설계
⭐⭐	도서 앱에 관리자 기능(책 추가·삭제)	books / auth / web, 파일 주인
⭐⭐	도서 앱 테스트 늘리기	모듈별 테스트 담당 + 블록 6 검문소
⭐⭐	이전 수업 프로젝트 보안·성능·테스트 리뷰	블록 3 역할 카드 재사용
⭐⭐⭐	내 프로젝트에 새 기능	직접 설계, 파일 소유표 필수

```
① 왜 팀이어야 하나? (혼자·심부름꾼으로 안 되는 이유 한 줄)
② 팀원: 이름 / 역할 / 모델 / 맡을 파일
③ 할 일 보드 초안: 팀원당 4~6개, 선행 작업
④ 결과물: 어떤 파일에 무엇이 남는가
⑤ 안전장치: 시작 전 커밋, 검문소(hook) 필요 여부
```

> 👩‍🏫 **강사 해설**
>
> ① 을 확인할 때는 오늘 본 근거로 판단합니다. 좋은 답은 "관점이 서로 달라서 반박이 필요하다(SQLite 찬반 조사)", "파일이 겹치지 않게 셋으로 나뉜다(관리자 기능: books / auth / web)"처럼 블록 2·3·5·7 에서 팀이 이긴 이유 중 하나를 댑니다. 약한 답은 "빨리 하려고", "많이 찾으려고"입니다. 블록 3 에서 팀이 더 많이 찾지는 않았다는 걸 이미 봤기 때문입니다.
>
> 메뉴별로 보면 이렇습니다. README·API 문서 리뷰는 관점이 셋이라 팀 연습은 되지만, 문서가 작아서 혼자로도 충분할 가능성이 큽니다. 비교해 보고 그 결론을 회고에 쓰게 하면 좋습니다. SQLite 조사는 찬성·반대가 서로 반박해야 결론이 나므로 팀에 잘 맞습니다. 관리자 기능은 블록 5 와 같은 구조라 파일 소유표만 지키면 팀이 맞습니다. 테스트 늘리기는 모듈별로 서로 대화할 필요가 적어서 심부름꾼 여러 명으로도 됩니다. 학생이 이걸 스스로 알아채면 오늘 목표를 달성한 것입니다.

8-2. 가동 (55분)

💬 클로드 대화창

```
/usage   # 시작 전 사용량 메모
(팀 설계서 내용) 으로 agent team 을 꾸릴 거야. 팀원별 업무 지시서를 먼저 보여주고, 내가 OK 하면 그때 뽑아.   # 블록 5-2 와 같은 순서
/usage   # 끝난 뒤 사용량 메모 → 회고에 사용
```

→ 10분마다 현황판(↓ Enter)·할 일 보드(Ctrl + T) 확인, 끝난 팀원은 바로 퇴근, 마지막 15분은 회고 메모 (팀이 정말 필요했나 / 엇나간 순간 / 사용량 대비 가치)

블록 9

발표 & 회고 (30분)

목표 — 학생 5~6명이 2분씩 화면을 공유하며 "왜 팀이어야 했나 / 팀원이 엇나간 순간과 바로잡은 방법 / 사용량 대비 가치"를 발표합니다. 서로의 판단을 들으며 팀을 쓸 때와 안 쓸 때의 기준을 정리합니다.

👩‍🏫 강사 해설

하루를 마무리할 때 오늘 실습이 보여 준 근거를 이렇게 묶어서 말해 주세요.

이런 일이면	이렇게	오늘의 근거
작고, 순서대로 이어지고, 한 파일을 고치는 일	🧑 혼자	블록 3: 작은 앱 리뷰는 혼자서도 13개를 다 찾았고 범위는 더 넓었다
과정은 길지만 결과만 필요한 일	🏃 심부름꾼	블록 1: Explore 는 파일 8개를 읽고 표만 돌려줘서 메인 대화가 깨끗했다
같은 일을 반복하고, 검사하는 눈이 따로 필요한 일	🏃 역할 카드 심부름꾼	블록 1·5: test-runner 를 오전에 만들고 오후에 팀의 결과를 검사했다
관점끼리 반박해야 결론이 나는 일	🏢 팀	블록 2: critic 의 반박으로 데이터 설계가 바뀌었다. 블록 4: 반박으로 추측이 재현으로 바뀌었다
"진짜로 일어나는지" 확인이 중요한 일	🏢 팀	블록 3: 같은 13개를 찾았지만 팀은 10개를 재현해서 23점 대 13점. 거짓 경보도 걸러 냈다
파일이 겹치지 않게 나눌 수 있는 큰 구현	🏢 팀 + 파일 주인	블록 5: books / auth / web / server.js 로 나눠 동시에 고쳤다
원인을 모르는 버그	🏢 팀 (경쟁 가설)	블록 7: 서로 다른 가설을 반박해서 닻 내림을 피했다

그리고 팀의 대가도 같이 정리합니다. 사용량은 팀원 수만큼 늘었고, 일을 나누면 경계에 빈틈이 생겼고(블록 3 의 범위 밖 항목), 팀장이 기다리지 않고 직접 손대거나 팀원이 금방 끝나 현황판에서 사라지는 등 운영에 손이 갑니다. 그래서 "왜 팀이어야 하나"에 위 표의 근거 중 하나로 답할 수 있을 때만 팀을 꾸린다는 것이 오늘의 결론입니다.